

Job Tracker Frontend Documentation

Covers application architecture, routing, authentication state, API layer, pages, components, views, animations, and drag-and-drop.

Table of Contents

1. [Executive Summary](#)
 2. [Architecture Overview](#)
 3. [Technology Stack](#)
 4. [Project Structure](#)
 5. [Routing](#)
 6. [Authentication State](#)
 7. [API Layer](#)
 8. [Pages](#)
 9. [Components](#)
 10. [Dashboard Views](#)
 11. [Animations](#)
 12. [Drag-and-Drop](#)
 13. [Constants](#)
 14. [Configuration and Environment](#)
-

1) Executive Summary

The Job Tracker frontend is a **React 19** single-page application (SPA) built with **Vite**. It communicates with the Express backend exclusively through a shared **Axios** instance that sends the auth cookie automatically on every request.

At a high level:

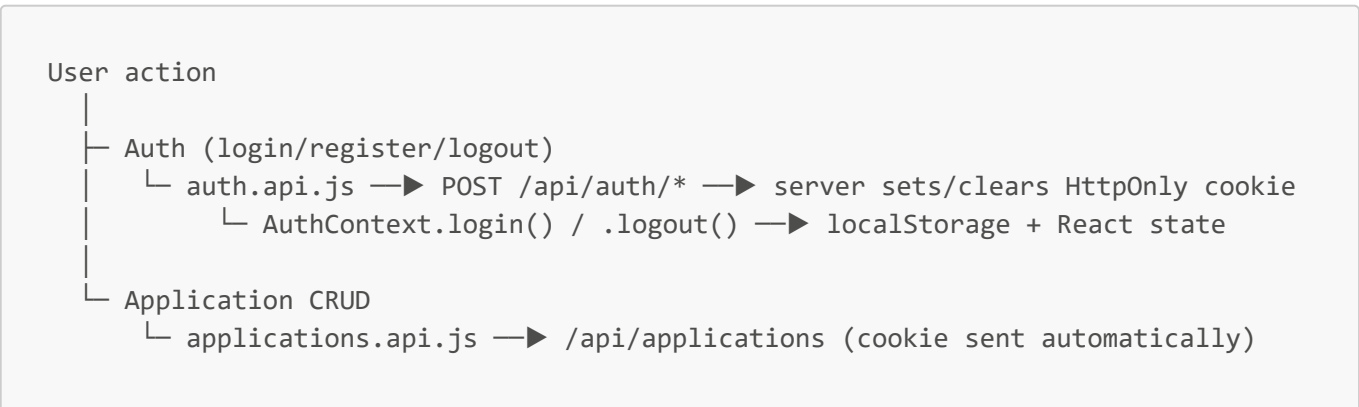
- **React Router DOM 7** manages client-side navigation between the login, register, and dashboard pages.
 - **React Context** (**AuthContext**) holds the global authentication state, initialized from **localStorage** on page load.
 - **Axios** (**withCredentials: true**) ensures the HttpOnly JWT cookie set by the server is included on every API call without any manual token handling.
 - The dashboard exposes three interchangeable views — **Cards**, **Table**, and **Kanban** — controlled by a tab switcher with a shared status filter dropdown.
 - **@dnd-kit** powers the Kanban board's drag-and-drop with an optimistic UI update strategy: the status change is reflected in React state immediately and then persisted to the backend asynchronously.
 - **GSAP** provides entrance animations on the auth pages and modal open/close transitions across the dashboard.
 - **Tailwind CSS 4** (loaded as a Vite plugin) handles all styling via utility classes, with a shared **constants/statuses.js** file supplying per-status color tokens used across all views.
-

2) Architecture Overview

2.1 Application Tree



2.2 Data Flow



└─ React state update in Applications.jsx

└─ Re-render propagated to active view

2.3 Key Design Decisions

- **No token in JavaScript** — The JWT lives exclusively in the HttpOnly cookie managed by the browser. `AuthContext` stores only `{ authenticated: true }` in `localStorage` as a lightweight UI flag to decide whether to show the dashboard or redirect to login — never the token itself.
- **Shared Axios instance** — All API modules import from `api/axios.js`. The single `withCredentials: true` setting there ensures cookies are sent on every request without each module needing to configure it.
- **Optimistic UI for Kanban** — Drag-and-drop status changes update React state first for an instant visual response, then persist to the backend. On failure, the state is resynced by refetching all applications.
- **Lift state to Applications** — All application data, filter state, view selection, and modal visibility live in the `Applications` page component. Views are pure presentational components that receive data and callbacks as props.

3) Technology Stack

3.1 Core Platform

Package	Version	Role
React	19.x	UI rendering and component model
Vite	7.x	Dev server, bundler, HMR
React DOM	19.x	DOM rendering target

3.2 Routing

Package	Version	Role
react-router-dom	7.x	Client-side routing, <code>BrowserRouter</code> , <code>Routes</code> , <code>Route</code> , <code>Navigate</code>

3.3 Styling

Package	Version	Role
Tailwind CSS	4.x	Utility-first CSS framework
@tailwindcss/vite	4.x	Vite plugin — loads Tailwind without a PostCSS config file

3.4 HTTP Client

Package	Version	Role
axios	1.x	HTTP requests to the backend API

3.5 Animation

Package	Version	Role
gsap	3.x	Timeline-based animations (entrance, modal transitions)
@gsap/react	2.x	useGSAP hook — scoped, cleanup-safe GSAP integration for React

3.6 Drag-and-Drop

Package	Version	Role
@dnd-kit/core	6.x	DndContext, DragOverlay, pointer/touch sensors
@dnd-kit/sortable	10.x	useSortable, SortableContext — makes cards draggable within columns
@dnd-kit/utilities	3.x	CSS.Transform.toString — converts dnd-kit transform to CSS string

4) Project Structure

client/	
index.html	# Vite HTML entry – mounts <div id="root">
vite.config.js	# Vite config: React plugin + Tailwind CSS
plugin	
constants/	
statuses.js	# STATUSES array and statusColors Tailwind
class map	
src/	
main.jsx	# App entry: BrowserRouter > AuthProvider >
App	
App.jsx	# Thin wrapper – renders AppRoutes
index.css	# Global CSS and Tailwind base styles
api/	
axios.js	# Shared Axios instance (baseUrl,
withCredentials)	
auth.api.js	# registerUser, loginUser, logoutUser
applications.api.js	# getApplications, createApplication,
updateApplication, deleteApplication	
context/	
AuthContext.jsx	# AuthContext, AuthProvider – global auth
state + localStorage sync	
routes/	
AppRoutes.jsx	# Route definitions: /login, /register, /
(protected)	
components/	
ProtectedRoute.jsx	# Redirects to /login if user is not
authenticated	
Navbar.jsx	# Top navigation bar with logo and logout
button	
Stats.jsx	# Dashboard statistics summary (circles per
status)	

ApplicationForm.jsx	# Create new application form
ApplicationCard.jsx	# Card UI for a single application (card
view)	
ApplicationDetailsModal.jsx	# View/edit modal with GSAP open/close
animation	
pages/	
Login.jsx	# Login page with GSAP entrance animation
Register.jsx	# Register page with GSAP entrance animation
Dashboard.jsx	# Layout shell: Navbar + Applications
Applications.jsx	# Main data page: state, filters, view
switcher, modals	
views/	
ViewTabs.jsx	# Cards / Table / Kanban tab switcher
CardView.jsx	# Responsive card grid view
TableView.jsx	# Tabular view with Company / Role / Status
columns	
KanbanView.jsx	# DndContext wrapper and column grid
KanbanColumn.jsx	# Single droppable Kanban column (one per
status)	
KanbanCard.jsx	# Draggable Kanban card (useSortable)

5) Routing

File: [src/routes/AppRoutes.jsx](#)

The application defines three routes. All routes are rendered inside a `BrowserRouter` initialized in `main.jsx`.

Path	Component	Access
<code>/login</code>	<code>Login</code>	Public
<code>/register</code>	<code>Register</code>	Public
<code>/</code>	<code>Dashboard</code> (wrapped in <code>ProtectedRoute</code>)	Authenticated only

There is no catch-all or 404 route — unmatched paths render nothing.

5.1 ProtectedRoute

File: [src/components/ProtectedRoute.jsx](#)

A wrapper component that reads `user` from `AuthContext`. If `user` is truthy (the `{ authenticated: true }` object is present in state), it renders `children`. Otherwise, it renders `<Navigate to="/login" />`, immediately redirecting the browser without rendering the protected page.

ProtectedRoute
└─ user exists → render children (Dashboard)
└─ user is null → <Navigate to="/login" />

6) Authentication State

File: `src/context/AuthContext.jsx`

6.1 Design

`AuthContext` holds a lightweight auth flag, not user profile data or the JWT. The actual token is managed entirely by the browser via the `HttpOnly` cookie set by the server. The context's sole responsibility is telling the UI whether the user is authenticated.

6.2 State Shape

```
// Stored in localStorage under the key "authState"
// and mirrored into React state as `user`
{ authenticated: true }
```

On page load, `AuthProvider` initializes `user` from `localStorage.getItem("authState")`. If the key is absent or null, `user` starts as `null` and the app routes to `/login`.

6.3 Context API

Value	Type	Description
<code>user</code>	<code>{ authenticated: true }</code> or <code>null</code>	Current auth state — truthy means authenticated
<code>login(data)</code>	<code>(data) => void</code>	Sets <code>{ authenticated: true }</code> in both <code>localStorage</code> and React state
<code>logout()</code>	<code>() => void</code>	Calls <code>logoutUser()</code> (fire-and-forget to clear server cookie), then removes <code>localStorage</code> entry and sets <code>user</code> to <code>null</code>

6.4 Auth Flow

Login / Register:

```
form submit → loginUser() / registerUser() API call
→ server validates credentials, sets HttpOnly cookie, returns 200
→ AuthContext.login() called
  → localStorage.setItem("authState", '{"authenticated":true}')
  → setUser({ authenticated: true })
→ navigate("/")
```

Logout:

```
logout button click → AuthContext.logout()
→ logoutUser() (POST /api/auth/logout, fire-and-forget)
```

```
→ server clears HttpOnly cookie
→ localStorage.removeItem("authState")
→ setUser(null)
→ ProtectedRoute detects null user → redirects to /login
```

Page refresh:

```
main.jsx mounts → AuthProvider initializes
→ JSON.parse(localStorage.getItem("authState"))
→ user = { authenticated: true } (if previously logged in)
→ ProtectedRoute passes → Dashboard renders
→ Applications fetches data (cookie sent automatically by browser)
```

7) API Layer

7.1 Axios Instance

File: [src/api/axios.js](#)

A single Axios instance is created and exported for use by all API modules.

Setting	Value	Purpose
<code>baseUrl</code>	<code>VITE_API_URL</code> env var, fallback <code>http://localhost:5000/api</code>	All relative paths in API modules resolve against this
<code>withCredentials</code>	<code>true</code>	Tells the browser to include the HttpOnly cookie on cross-origin requests

7.2 Auth API

File: [src/api/auth.api.js](#)

Function	Method	Endpoint	Description
<code>registerUser(formData)</code>	POST	<code>/auth/register</code>	Creates a new account; server sets auth cookie on success
<code>loginUser(formData)</code>	POST	<code>/auth/login</code>	Authenticates user; server sets auth cookie on success
<code>logoutUser()</code>	POST	<code>/auth/logout</code>	Signals server to clear auth cookie

7.3 Applications API

File: [src/api/applications.api.js](#)

Function	Method	Endpoint	Returns
----------	--------	----------	---------

Function	Method	Endpoint	Returns
<code>getApplications()</code>	GET	<code>/applications</code>	Array of all application objects for the current user
<code>createApplication(data)</code>	POST	<code>/applications</code>	The newly created application object
<code>updateApplication(id, data)</code>	PUT	<code>/applications/:id</code>	The updated application object
<code>deleteApplication(id)</code>	DELETE	<code>/applications/:id</code>	No return value (<code>void</code>)

All calls inherit `withCredentials: true` from the shared Axios instance, so no manual token passing is needed.

8) Pages

8.1 Login

File: `src/pages/Login.jsx`

Renders the login form. On mount, runs a GSAP entrance timeline (card scale-in, title/subtitle slide-up, input stagger, button pop). On submit, calls `loginUser()`, then `AuthContext.login()`, then navigates to `/`. On API error, displays the server's error message in a styled inline error banner.

State	Type	Description
<code>form</code>	<code>{ email, password }</code>	Controlled form fields
<code>error</code>	<code>string</code>	Inline error message from the server

8.2 Register

File: `src/pages/Register.jsx`

Identical structure to `Login`. Renders a registration form with an additional Name field. Runs the same GSAP entrance animation on mount. On submit, calls `registerUser()`, then `AuthContext.login()`, then navigates to `/`.

State	Type	Description
<code>form</code>	<code>{ name, email, password }</code>	Controlled form fields
<code>error</code>	<code>string</code>	Inline error message from the server

8.3 Dashboard

File: `src/pages/Dashboard.jsx`

Layout shell. Renders `<Navbar />` at the top and `<Applications />` inside a padded `<main>` element. Holds no state of its own.

8.4 Applications

File: [src/pages/Applications.jsx](#)

The primary data page. Owns all application state and orchestrates the dashboard UI.

State	Type	Description
<code>apps</code>	<code>Application[]</code>	Full list fetched from the server
<code>loading</code>	<code>boolean</code>	Controls the loading text while data is being fetched
<code>view</code>	<code>"cards" "table" "kanban"</code>	Active dashboard view
<code>filter</code>	<code>string</code>	Selected status filter (" <code>All</code> " or a specific status)
<code>showForm</code>	<code>boolean</code>	Whether the New Application modal is visible

Key behaviors:

- `loadApplications()` — fetches all applications from the server and sets `apps`. Called on mount and after any create/update/delete.
- `filteredApps` — derived from `apps` by filtering on `filter`. Passed to the active view.
- `updateStatus(id, status)` — updates `apps` in React state immediately (optimistic), then calls `updateApplication(id, { status })`. On failure, calls `loadApplications()` to resync.
- When `showForm` becomes `true`, a GSAP animation fades in the backdrop overlay and scales in the modal card.
- Clicking the backdrop overlay (outside the modal card) closes the form via event propagation guard (`e.stopPropagation()` on the inner div).

9) Components

9.1 Navbar

File: [src/components/Navbar.jsx](#)

Top navigation bar. Reads `user` and `logout` from `AuthContext`. Renders the app logo on the left and, if a user is authenticated, the user's name and a Logout button on the right. Clicking Logout calls `AuthContext.logout()`.

9.2 Stats

File: [src/components/Stats.jsx](#)

Displays a row of circular stat badges — one for Total and one per status. Computes counts by reducing over the `applications` prop array. Only rendered when `apps.length > 0`.

Prop	Type	Description
<code>applications</code>	<code>Application[]</code>	Full (unfiltered) application list

Internally uses a private `StatCircle` sub-component that renders a labeled colored circle with the count inside.

9.3 `ApplicationForm`

File: `src/components/ApplicationForm.jsx`

A controlled form for creating a new job application. Rendered inside the New Application modal in `Applications`. On successful submit, resets all fields to their defaults and calls the `onAdd` callback, which triggers a data reload and closes the modal.

Prop	Type	Description
<code>onAdd</code>	<code>() => void</code>	Called after a successful create — triggers <code>loadApplications()</code> and closes the modal

Field	Required	Default
Company	Yes	""
Role	Yes	""
Job Link	No	""
Notes	No	""
Follow Up Date	No	""

9.4 `ApplicationCard`

File: `src/components/ApplicationCard.jsx`

A card tile used in the Card View. Displays the company name, status pill (colored via `statusColors`), and role. Provides a **View** button (calls `onView` — opens `ApplicationDetailsModal` in the parent) and a **Delete** button (shows `window.confirm`, then calls `deleteApplication` and triggers `onChange`).

Prop	Type	Description
<code>app</code>	<code>Application</code>	The application object to display
<code>onChange</code>	<code>() => void</code>	Called after delete — triggers <code>loadApplications()</code>
<code>onView</code>	<code>() => void</code>	Called on View button click — sets <code>selectedApp</code> in the parent

9.5 `ApplicationDetailsModal`

File: `src/components/ApplicationDetailsModal.jsx`

A full-detail view/edit modal for a single application. Used by both `CardView` and `TableView`. On mount, GSAP animates the backdrop to opacity 1 and scales the modal card in. On close, GSAP animates the card out (fade + slide down) and then calls `onClose` in the `onComplete` callback.

The modal has two modes controlled by local `editing` state:

Mode	Behavior	
View	Displays all fields as read-only text. Status is shown as a colored label. Job link renders as a clickable anchor with <code>target="_blank"</code> . Follow-up date is formatted with <code>toLocaleDateString</code> (UTC timezone to avoid off-by-one day issues).	
Edit	All fields become inputs/selects/textareas. Status becomes a <code><select></code> populated from <code>STATUSES</code> . Saving calls <code>updateApplication(app._id, form)</code> , then calls <code>onChange?.()</code> and closes the modal.	
Prop	Type	Description
<code>app</code>	<code>Application</code>	The application to display / edit
<code>onClose</code>	<code>() => void</code>	Called after the close animation completes
<code>onChange</code>	<code>() => void</code> (optional)	Called after a successful save — triggers <code>loadApplications()</code>

10) Dashboard Views

All three views receive `applications` (the filtered list from `Applications`) as a prop. None of them fetch data — they are purely presentational.

10.1 ViewTabs

File: [src/views/ViewTabs.jsx](#)

A tab bar with three buttons: **Cards**, **Table**, **Kanban**. The active tab is underlined and bold. Clicking a tab calls `setView` with the tab's ID.

Prop	Type	Description
<code>view</code>	<code>string</code>	Currently active view ID
<code>setView</code>	<code>(id: string) => void</code>	Updates view state in <code>Applications</code>

10.2 CardView

File: [src/views/CardView.jsx](#)

Renders a responsive CSS grid (`md:grid-cols-2 lg:grid-cols-3`) of `ApplicationCard` components. Manages `selectedApp` state locally to control `ApplicationDetailsModal` visibility. Clicking **View** on a card sets `selectedApp`; closing the modal sets it back to `null`.

Prop	Type	Description
<code>applications</code>	<code>Application[]</code>	Filtered list to render
<code>onChange</code>	<code>() => void</code>	Passed through to cards and modal for post-action data reload

10.3 TableView

File: [src/views/TableView.jsx](#)

Renders a full-width table with three columns: **Company**, **Role**, **Status**. The status cell is a clickable colored pill — clicking it opens `ApplicationDetailsModal` for that row. Manages `selectedApp` locally for modal visibility.

Uses an internal `TableRow` sub-component to keep the row render logic self-contained.

Prop	Type	Description
<code>applications</code>	<code>Application[]</code>	Filtered list to render

10.4 KanbanView

File: `src/views/KanbanView.jsx`

Wraps the board in a `DndContext`. Groups applications by status into four columns using `STATUSES.reduce`. Renders one `KanbanColumn` per status.

Drag lifecycle:

Event	Handler	Action
<code>onDragStart</code>	<code>handleDragStart</code>	Finds the dragged application in <code>applications</code> by <code>active.id</code> and sets it as <code>activeApp</code> for the <code>DragOverlay</code>
<code>onDragEnd</code>	<code>handleDragEnd</code>	Reads <code>over.data.current.status</code> from the drop target column; calls <code>onStatusChange(active.id, newStatus)</code> — a no-op if dropped outside any column

A `DragOverlay` renders a ghost card (scaled up with elevated shadow) while dragging.

Prop	Type	Description
<code>applications</code>	<code>Application[]</code>	Filtered list — all statuses
<code>onStatusChange</code>	<code>(id, status) => void</code>	<code>Applications.updateStatus</code> — performs optimistic update then persists

10.5 KanbanColumn

File: `src/views/KanbanColumn.jsx`

A droppable zone for one status. Uses `useDroppable` from `@dnd-kit/core` with `data: { status }` so `KanbanView.handleDragEnd` can read the target status from `over.data.current`. Wraps cards in `SortableContext` with `verticalListSortingStrategy` to enable animated reordering feedback within the column.

Prop	Type	Description
<code>status</code>	<code>string</code>	The status this column represents
<code>applications</code>	<code>Application[]</code>	Cards belonging to this status

10.6 KanbanCard

File: `src/views/KanbanCard.jsx`

A draggable card within a Kanban column. Uses `useSortable({ id: app._id })` to get drag attributes, listeners, transform, and transition values. Applies `CSS.Transform.toString(transform)` as an inline style for smooth drag movement. Sets `opacity: 0.4` while `isDragging` is true (the card's original position while the ghost follows the cursor).

Prop	Type	Description
<code>app</code>	<code>Application</code>	The application to display — shows company name and role

11) Animations

All animations use **GSAP** (GreenSock Animation Platform). The `@gsap/react` package provides the `useGSAP` hook, which scopes animations to a container ref and automatically cleans up on component unmount.

11.1 Auth Page Entrance (`Login`, `Register`)

Both pages run an identical staggered entrance timeline on mount, scoped to a `containerRef` wrapping the page:

Step	Target	Animation	Duration	Ease
1	<code>.auth-card</code>	Scale from 0.9 + fade in	0.8s	<code>power3.out</code>
2	<code>.auth-title</code> , <code>.auth-subtitle</code>	Slide up 30px + fade in (stagger 0.1s)	0.6s	<code>power3.inOut</code>
3	<code>.auth-input</code> elements	Slide up 20px + fade in (stagger 0.08s)	0.6s	<code>back.out(1.7)</code>
4	<code>.auth-button</code>	Scale from 0.9 + fade in	0.4s	<code>back.out(1.7)</code>

The timeline uses a 0.2s initial delay and overlapping offsets (`"-=0.4"`, `"-=0.2"`) for a fluid feel.

11.2 New Application Modal (`Applications`)

Triggered imperatively with `useEffect` when `showForm` becomes `true`:

Target	Animation
Backdrop overlay	Fade in opacity 0 → 1, 0.2s
Modal card	Scale 1 + opacity 1, 0.5s, <code>power3.out</code>

No close animation — the modal is removed from the DOM by setting `showForm = false`.

11.3 Application Details Modal (`ApplicationDetailsModal`)

Open and close animations are triggered imperatively with `useEffect` and `handleClose`:

Trigger	Target	Animation
---------	--------	-----------

Trigger	Target	Animation
Mount	Backdrop	Fade in, 0.2s
Mount	Modal card	Scale 1 + opacity 1, 0.5s, <code>power3.out</code>
Close	Modal card	Opacity 0 + slide down 30px + scale 0.97, 0.2s; <code>onClose</code> called in <code>onComplete</code>
Close	Backdrop	Fade out, 0.2s

The close animation completes before the component is unmounted, preventing a jarring instant removal.

12) Drag-and-Drop

The Kanban board uses **@dnd-kit** for accessible, pointer- and touch-compatible drag-and-drop.

12.1 Component Roles

Component	@dnd-kit Role	Hook / Component Used
<code>KanbanView</code>	Drag context provider	<code>DndContext</code> , <code>DragOverlay</code>
<code>KanbanColumn</code>	Drop target	<code>useDroppable</code>
<code>KanbanCard</code>	Draggable item	<code>useSortable</code>

12.2 Drag-and-Drop Flow

```
User grabs KanbanCard
└─ DndContext fires onDragStart
    └─ handleDragStart: find app by active.id → setActiveApp(app)
        └─ DragOverlay renders ghost card (scale-105, shadow-2x1)

User moves card over KanbanColumn
└─ @dnd-kit tracks pointer position
└─ KanbanCard (original) renders at opacity 0.4

User drops card
└─ DndContext fires onDragEnd({ active, over })
    ├── setActiveApp(null) – remove ghost overlay
    ├── if !over → return (dropped outside board)
    ├── newStatus = over.data.current.status
    └─ onStatusChange(active.id, newStatus)
        └─ Applications.updateStatus(id, newStatus)
            ├── setApps(... optimistic update ...) – instant UI
            └─ updateApplication(id, { status }) – persist to server
                └─ on error: loadApplications() – resync fallback
```

12.3 Column Drop Target Data

Each `KanbanColumn` registers a droppable with `data: { status }`. This is the mechanism by which `handleDragEnd` knows which status to apply — it reads `over.data.current.status` rather than parsing the column ID string.

13) Constants

File: `constants/statuses.js`

Shared between multiple components to keep status values and their color mappings consistent across all views.

STATUSES

An ordered array of all valid application statuses. Used to:

- Populate the status `<select>` in `ApplicationDetailsModal`
- Define the four Kanban columns in `KanbanView`

```
[ "Applied", "Interviewing", "Offer", "Rejected" ]
```

statusColors

A map of status string → Tailwind CSS utility classes. Used to apply consistent color styling to status pills, Kanban column headers, and stat circles across `ApplicationCard`, `TableView`, `KanbanColumn`, `ApplicationDetailsModal`, and `Stats`.

Status	Color
Applied	Blue (<code>text-blue-500 border-blue-500 hover:bg-blue-500/10</code>)
Interviewing	Yellow (<code>text-yellow-500 border-yellow-500 hover:bg-yellow-500/10</code>)
Offer	Green (<code>text-green-500 border-green-500 hover:bg-green-500/10</code>)
Rejected	Red (<code>text-red-500 border-red-500 hover:bg-red-500/10</code>)

14) Configuration and Environment

Environment Variables

Variable	Description
<code>VITE_API_URL</code>	Full base URL for the backend API (e.g. <code>http://localhost:5000/api</code>). Falls back to <code>http://localhost:5000/api</code> if not set.

Create a `.env` file in the `client/` directory:

```
VITE_API_URL=http://localhost:5000/api
```

All Vite environment variables must be prefixed with `VITE_` to be accessible in client-side code via `import.meta.env`.

Development Commands

```
# Install dependencies
npm install

# Start dev server with HMR (default port: 5173)
npm run dev

# Build for production
npm run build

# Preview the production build locally
npm run preview

# Lint
npm run lint
```

Production Considerations

- Set `VITE_API_URL` to the deployed backend URL during the production build.
- The Axios instance uses `withCredentials: true`, so the backend must set `Access-Control-Allow-Credentials: true` and the `Access-Control-Allow-Origin` header must specify the exact frontend origin (not `*`).
- GSAP is included in the production bundle. If bundle size is a concern, only the required GSAP modules are used (core + `@gsap/react` hook).

15) Component-by-Component Reference

`client/src/main.jsx`

Entry point. Creates the React root and renders `BrowserRouter > AuthProvider > App`. Imports `index.css` for global styles and Tailwind base.

`client/src/App.jsx`

Thin wrapper component — renders `<AppRoutes />` with no additional logic or state.

`client/src/routes/AppRoutes.jsx`

Defines all three routes using `react-router-dom`. The root route wraps `Dashboard` in `ProtectedRoute` to enforce authentication.

`client/src/context/AuthContext.jsx`

Creates `AuthContext` and `AuthProvider`. Initializes state from `localStorage` so the user remains logged in across page refreshes. Exposes `user`, `login`, and `logout`. The `logout` function calls `logoutUser()` without awaiting it — if the network call fails, the client still clears local state and the user is redirected.

`client/src/api/axios.js`

Exports a single configured Axios instance. The `baseUrl` and `withCredentials` settings are the only two configuration points — all API modules inherit them by importing this instance.

`client/src/api/auth.api.js`

Three functions wrapping the auth endpoints. Each function returns `res.data` so callers receive only the response body, not the full Axios response object.

`client/src/api/applications.api.js`

Four CRUD functions for the applications resource. `deleteApplication` returns `void` — no response body is expected from the server.

`client/src/components/ProtectedRoute.jsx`

Single-purpose guard component. No state or effects — purely reads `user` from `AuthContext` and returns either `children` or a redirect.

`client/src/components/Navbar.jsx`

Reads `user` and `logout` from `AuthContext`. Conditionally renders the user name and logout button only when `user` is truthy.

`client/src/components/Stats.jsx`

Computes status counts from the `applications` prop using a single `reduce`. Uses an internal `StatCircle` component for consistent circle rendering. References `statusColors` from `constants/statuses.js` for the color of each circle.

`client/src/components/ApplicationForm.jsx`

Fully controlled form with local state. On submit, calls `createApplication(form)`, resets the form to default values, and calls `onAdd()`. Does not manage its own visibility — that is handled by the `Applications` page.

`client/src/components/ApplicationCard.jsx`

Presentational card with two action buttons. Delete requires `window.confirm` confirmation before calling the API. Both `onChange` and `onView` callbacks are driven by the parent (`CardView`).

`client/src/components/ApplicationDetailsModal.jsx`

The most complex component. Manages local `editing` state and a mirrored `form` state initialized from `app` props. GSAP open/close animations are imperative (triggered in `useEffect` and `handleClose`). The close sequence completes the animation before calling `onClose` via the GSAP `onComplete` callback to prevent the component from unmounting before the animation finishes.

`client/src/pages/Login.jsx` and `client/src/pages/Register.jsx`

Auth pages with identical structure and animation. Each scopes its GSAP timeline to a `containerRef` via `useGSAP`. On successful API call, each invokes `AuthContext.login()` and uses `useNavigate` to redirect to `/`.

`client/src/pages/Dashboard.jsx`

Layout-only page. Composes `Navbar` and `Applications`. No state.

`client/src/pages/Applications.jsx`

Central state hub for the dashboard. Owns `apps`, `loading`, `view`, `filter`, and `showForm`. Provides `loadApplications` as a refresh callback to all child components that mutate data. Derives `filteredApps` from `apps + filter` without storing the filtered result in state.

`client/src/views/ViewTabs.jsx`

Pure presentational component. Renders tab buttons and applies active styling based on the `view` prop. Calls `setView` on click.

`client/src/views/CardView.jsx`

Manages `selectedApp` locally to control modal visibility. Passes `onView` to each `ApplicationCard` and `onClose` / `onChange` to `ApplicationDetailsModal`.

`client/src/views/TableView.jsx`

Contains an internal `TableRow` component for row rendering. Status pill is clickable to open the details modal. Manages `selectedApp` locally.

`client/src/views/KanbanView.jsx`

Orchestrates drag-and-drop. Groups applications into columns, handles `onDragStart` and `onDragEnd`, and renders a `DragOverlay` ghost card. Calls `onStatusChange` on successful drop, which triggers the optimistic update in `Applications`.

`client/src/views/KanbanColumn.jsx`

Registers as a droppable zone using `useDroppable` with `data: { status }`. Wraps cards in `SortableContext` for animated drag feedback within the column.

`client/src/views/KanbanCard.jsx`

Registers as a draggable item using `useSortable`. Applies CSS transform and transition from the hook for smooth drag motion. Reduces opacity to 0.4 at the card's original position while it is being dragged.

`client/constants/statuses.js`

Single source of truth for status values and their color mappings. Imported by `Stats`, `ApplicationCard`, `ApplicationDetailsModal`, `KanbanView`, `KanbanColumn`, and `TableView`.